University of Asia Pacific

Department of Computer Science and Engineering

CSE 438: Robotics Lab

Smart Automatic Plant Watering Robotic Car

Final Project Report

Project Title	Smart Automatic Plant Watering Robotic Car
Course	CSE 438: Robotics Lab
Team Name	TriBots
Team Members	Tanjina Akter Nisa (ID: 22201086) Md. Daudul Islam (ID: 22201088) Saswata Ghosal (ID: 22201090)
Submitted To	A S Zaforullah Momtaz Assistant Professor, CSE, UAP
Submission Date	12 May 2026

Department of Computer Science and Engineering

Faculty of Science and Technology
University of Asia Pacific

Dhaka, Bangladesh

Contents

List of Figures

List of Tables

Abstract

This report presents the design, development, and testing of a **Smart Automatic Plant Watering Robotic Car**, developed as a final CEP project for *CSE 438: Robotics Lab* at the University of Asia Pacific. The robot can automatically follow a predefined black line path, detect plant stop points using IR sensors, measure soil moisture with a servo-based sensing system, and activate a water pump through a relay module using an ESP32 microcontroller.

The project combines embedded systems, sensor calibration, motor control, and power management in a compact robotic platform. It was developed to solve a common problem: maintaining regular plant care during busy schedules or long absences.

During development, several hardware issues were faced, including IR sensor calibration problems, ESP32 signal errors, relay overheating, water pump damage from direct voltage supply, and voltage mismatches between 3.3,V and 5,V components. These issues were identified and solved through testing and troubleshooting, providing valuable practical experience.

The final prototype achieved successful autonomous operation. Two user surveys — one before development and one after completion — showed both the real need for the system and its effectiveness as a low-cost smart agriculture solution.

1. Introduction

Taking care of plants needs regular attention, which is often difficult for people with busy schedules, studies, or travel. For home gardeners, especially those with indoor or balcony plants, missing a few days of watering can easily damage the plants. This is a common problem in daily life.

Most plant watering is done manually and depends on human memory and availability. During absence, people may ask others to water the plants, but they may not know the correct amount. This can lead to overwatering or underwatering, both of which can harm plant health.

Figure 1: Complete assembled prototype — Smart Automatic Plant Watering Robotic Car

With the advancement of embedded systems and microcontroller technology, automated plant care systems have become more practical and affordable. This project, the Smart Automatic Plant Watering Robotic Car, is designed to solve this issue. The robot follows a black line path, stops at each plant location, checks soil moisture, and waters the plant only when needed. It uses an ESP32 microcontroller, IR sensors for navigation and stopping, a servo motor, a relay-controlled water pump, and a 4WD robotic chassis.

All components were chosen based on low cost and local availability. This report explains the full development process from the initial idea to the final working prototype, including results and challenges during implementation.

2. Problem Statement

The main problem addressed by this project is the unreliability of manual plant watering in daily life. The key issues are:

- People often forget to water plants during busy schedules, which can cause plants to dry out and die.
- During travel or long absences, plant care is usually given to others who may not know the correct watering amount or timing.
- Both overwatering and underwatering can harm plant health. Timer-based systems cannot check real soil conditions and may water unnecessarily.
- Many existing automated systems are either expensive or fixed in one place, making them unsuitable for low-cost multi-plant setups.

The challenge is to design a low-cost autonomous robot that can move to multiple plant stations, measure soil moisture, and decide when to water without human involvement.

3. Objectives

1. Design and develop an autonomous line-following robot using three IR sensors.
2. Implement plant stop-point detection using combined sensor input logic.
3. Integrate a servo-based soil moisture sensing system for accurate soil condition measurement.
4. Develop conditional watering logic so the water pump activates only when the soil is dry.
5. Build the system using affordable and locally available components within a limited budget.

6. Document the real implementation problems and their solutions as part of the engineering process.

4. Literature Review

4.1 Automated Irrigation Systems

Dastider and Hossain (2019) demonstrated a basic automated plant watering system using an Arduino microcontroller connected to a soil moisture sensor and a relay-controlled pump. Their system activated watering only when soil conductivity dropped below a set threshold. However, their implementation was stationary and served a single plant location.

Patel and Shah (2021) extended this concept with IoT integration using an ESP8266 to allow remote monitoring via a mobile application. Their work highlighted the advantage of ESP-series microcontrollers for smart agriculture applications due to built-in wireless connectivity and flexible GPIO configurations.

4.2 Line-Following Robotics

Line-following robots using IR sensors are well established in educational robotics. The standard approach uses reflective IR sensors to detect contrast between black and white surfaces, with correction steering applied based on left–right sensor deviations. This technique is reliable on smooth, flat surfaces and has been widely validated in conveyor-style automation.

4.3 Integration Challenges

Several research papers note the difficulty of integrating mobile robotics with sensing and actuation systems when power budgets are limited. Signal level mismatches between 3.3 V microcontrollers (such as the ESP32) and 5 V peripheral modules are frequently documented as a source of reliability issues. This was directly experienced during our project and is discussed in detail in Section 13.

4.4 Research Gap

Most existing systems are either stationary irrigation systems or mobile robots without irrigation capability. A mobile robot that actively checks soil moisture and waters multiple plant locations autonomously represents a practical combination that has not been widely implemented at low cost. This is the gap our project addresses.

5. System Design

The system is designed using four main subsystems that work together to perform automatic plant watering: Line Following System, Soil Moisture Sensing System, Watering Mechanism, and Mobility System.

All subsystems are controlled and coordinated by the ESP32 microcontroller, which acts as the central processing unit of the robot. The complete hardware architecture, including power distribution, motor control, sensor integration, and actuator connections, is illustrated in the circuit diagram below.

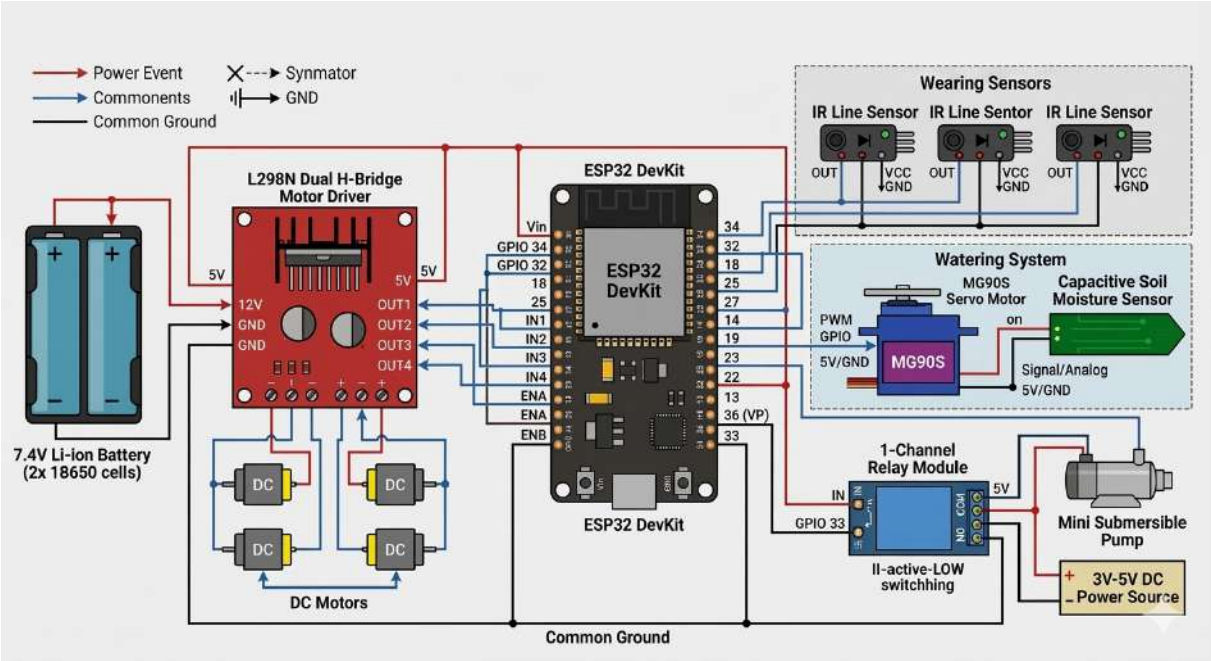


Figure 2: Complete Circuit Diagram of the Smart Automatic Plant Watering Robotic Car

The circuit diagram shows the integration of the ESP32 DevKit with L298N motor driver, three IR line sensors, capacitive soil moisture sensor, MG90S servo motor, relay module, and mini submersible water pump. Proper power management using 7.4V Li-ion battery and voltage distribution is also clearly depicted.

5.1 Navigation Subsystem

Three IR sensors are mounted at the front-bottom of the chassis. These sensors detect the black line on the ground and help the robot follow the correct path. The left and right sensors are used for steering correction, while the middle sensor ensures forward movement.

When the robot reaches a plant station, all three sensors detect a wide black mark. This signal is sent to the ESP32, which identifies it as a stopping point for moisture checking and watering.

The navigation subsystem ensures stable movement along the path and accurate stopping at each plant location.

Figure 3: Navigation subsystem using IR sensors

5.2 Moisture Detection Subsystem

A servo motor is mounted at the front section of the robotic chassis with the soil moisture sensor attached to its arm. When the robot stops at a plant station, the servo rotates downward and inserts the sensor probe into the soil.

After a short stabilisation delay, the ESP32 reads the analog moisture value from the sensor and compares it with a predefined threshold value. Based on the reading, the system determines whether watering is required.

This subsystem enables automatic soil condition monitoring without human intervention.

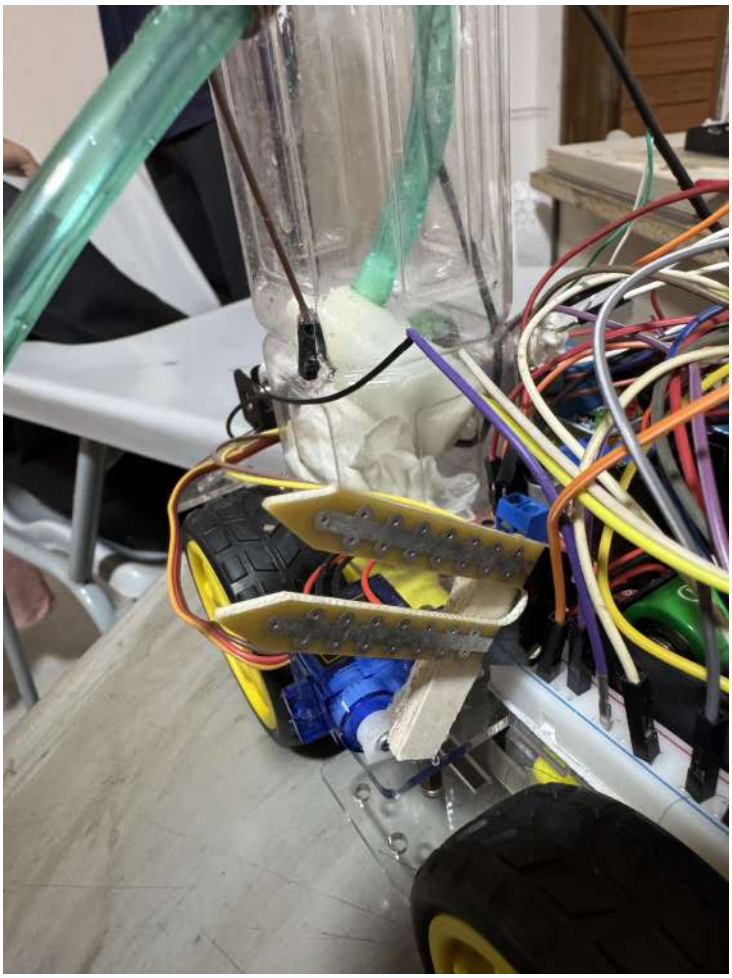


Figure 4: Moisture detection subsystem

5.3 Watering Control Subsystem

The watering mechanism is controlled using a relay module connected to the ESP32. The relay acts as an electronic switch for controlling the DC water pump.

If the ESP32 detects dry soil conditions, it activates the relay and turns on the water pump for a fixed duration. Water is then supplied from the onboard reservoir through a pipe toward the target plant.

When adequate soil moisture is detected, the relay remains inactive and watering is skipped, preventing unnecessary water usage.

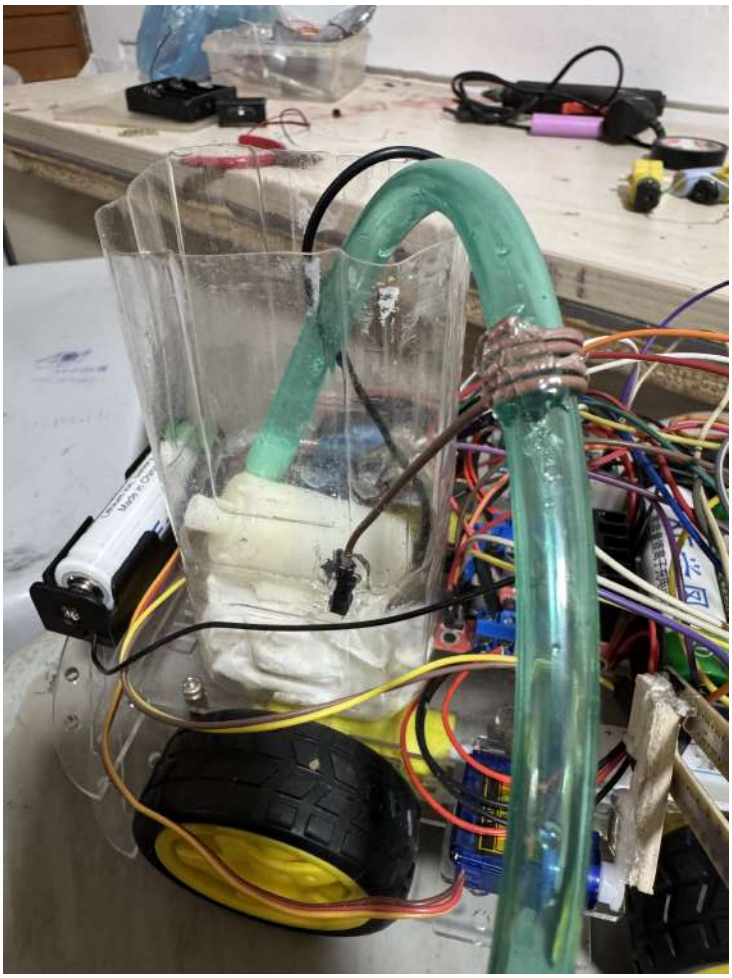


Figure 5: Watering control subsystem

5.4 Power Management

The motors and water pump require significantly higher current compared to the ESP32 microcontroller and sensors. To ensure stable operation, the system uses separate regulated power distribution for control electronics and high-current loads.

The ESP32 and sensors are powered using a stable regulated supply, while the motor driver and water pump operate from a separate battery circuit. This electrical isolation helps prevent voltage fluctuations caused by motor load changes.

Proper power management improves system reliability and protects sensitive electronic components from unexpected resets or unstable operation.

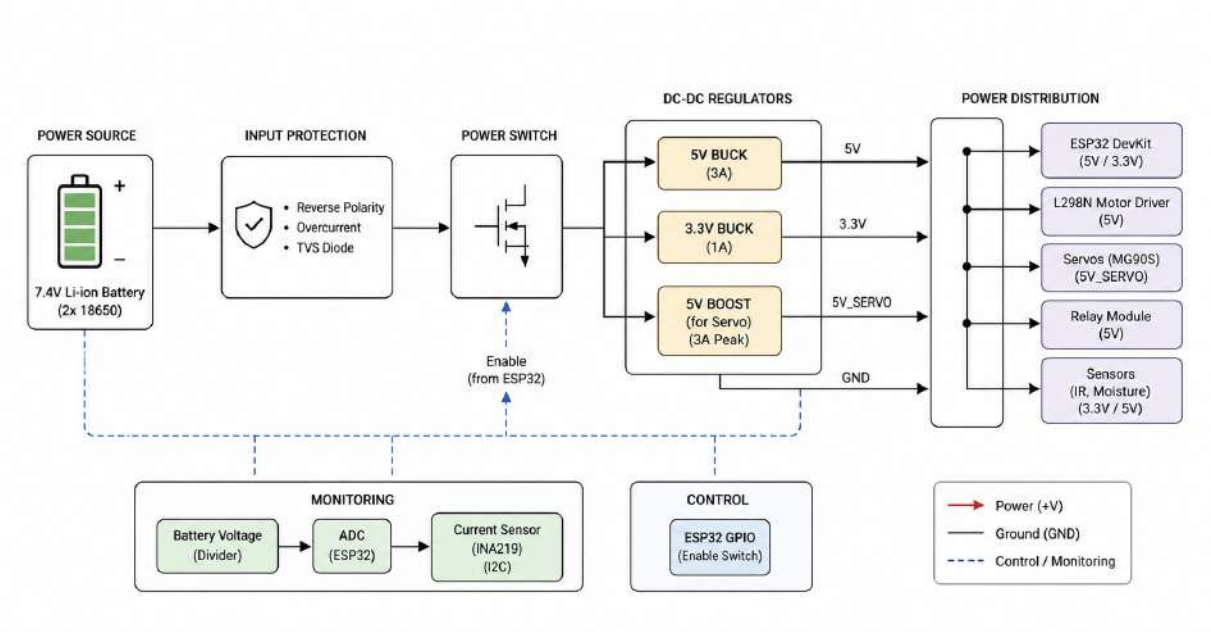


Figure 6: Power distribution and management setup

6. Components Description

All major hardware components used in the project are listed in Table 1.

Table 1: Hardware components used in the project

No.	Component	Description	Qty
1	ESP32 Microcontroller	Main control unit with dual-core processor, built-in Wi-Fi and Bluetooth capability.	1
2	L298N Motor Driver	Dual H-bridge module that controls direction and speed of DC motors via PWM signals.	1
3	4WD Robotic Chassis	Mechanical base platform of the robot with four motors and wheels.	1 set
4	IR Sensors	Reflective infrared sensors for line tracking, navigation, and station detection.	3
5	Servo Motor (SG90)	Micro servo that physically moves the soil moisture sensor up and down.	1
6	Soil Moisture Sensor	Resistive probe sensor that measures electrical conductivity of soil to detect moisture.	1
7	Relay Module	Electromechanical switch controlled by the ESP32 to turn the water pump ON or OFF.	1
8	Water Pump (DC)	Submersible DC mini pump that draws water from the reservoir and delivers it to plants.	1
9	Battery Pack	Rechargeable power supply that powers the entire robot system.	1

6.1 ESP32 Microcontroller

The ESP32 acts as the main controller of the system, handling all inputs and outputs. It has a dual-core processor, built-in Wi-Fi and Bluetooth, and a 12-bit ADC for reading sensors like the soil moisture sensor. Its GPIO pins are used to control the motor driver, servo motor, and relay.

The ESP32 was chosen over Arduino because of its higher processing power, more built-in features, and easy support for future IoT expansion without extra hardware.

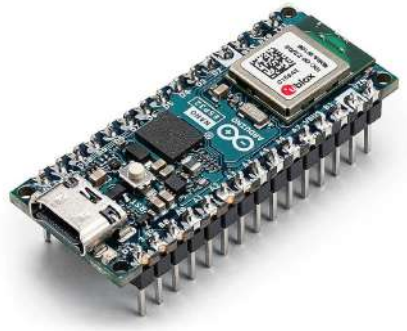


Figure 7: ESP32 Microcontroller

6.2 L298N Motor Driver

The L298N is a dual H-bridge motor driver used to control two DC motors in both forward and reverse directions. It takes control signals from the ESP32 GPIO pins and safely drives higher-current motor circuits, protecting the microcontroller from back-EMF and current spikes. It also includes a built-in 5 V regulator for powering small components and supports motor voltages from 5 V to 46 V with up to 2 A current per channel.

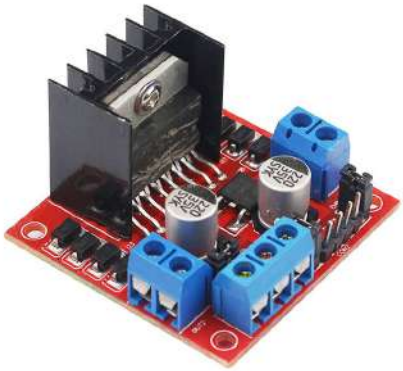


Figure 8: L298N Motor Driver

6.3 4WD Robotic Chassis

The 4WD chassis is the main structure of the robot, using four DC gear motors with rubber wheels for stable indoor movement. The acrylic body has pre-drilled holes for mounting all components like the ESP32, motor driver, sensors, and pump. Its two-layer design helps keep the centre of gravity low and improves stability during movement and turning.

Figure 9: 4WD Robotic Chassis

6.4 IR Sensors

Three IR sensors are mounted at the front underside of the chassis to detect the black line. They measure reflected infrared light, where white reflects more and black reflects less, sending signals to the ESP32. The left and right sensors control steering, while the middle sensor ensures straight movement. When all three detect a stop marker, the robot stops. Each sensor has a potentiometer for calibration.

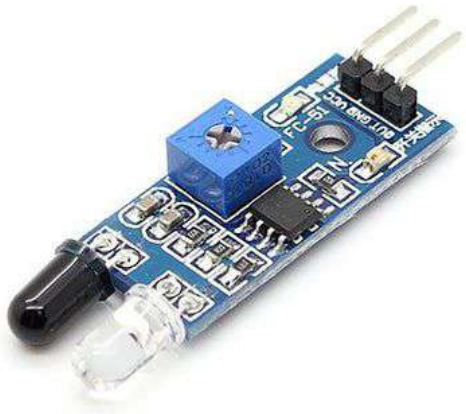


Figure 10: IR Sensors

6.5 Servo Motor (SG90)

The SG90 is a small 9 g micro servo motor with a 180° rotation range, controlled by PWM signals from the ESP32. It is used to move the soil moisture probe up and down at each plant station. Its small size, light weight, and sufficient torque make it suitable for this task. The servo position is controlled by PWM, with one angle for raised position and another for lowering the probe into the soil.

Figure 11: Servo Motor (SG90)

6.6 Soil Moisture Sensor

The soil moisture sensor uses two probes to measure electrical resistance in the soil. Wet soil gives lower resistance, while dry soil gives higher resistance, changing the output voltage. The ESP32 reads this value using its 12-bit ADC (0–4095) and compares it with a set threshold to decide when watering is needed. The sensor works with 3.3–5 V and provides both analog and digital outputs.

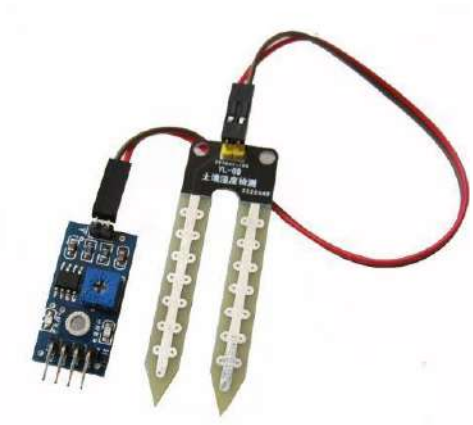


Figure 12: Soil Moisture Sensor

6.7 Relay Module

The relay module acts as an electronic switch that allows the ESP32 (3.3 V) to control the high-current water pump safely. It has a built-in transistor, so it can work directly with ESP32 signals. When the ESP32 sets the pin HIGH, the pump turns on; when LOW, it turns off. The 10 A rating ensures safe operation without overheating.

Figure 13: Relay Module

6.8 Water Pump

The submersible DC mini water pump operates at 3–6 V and is used to water small indoor plants. It is placed in a water reservoir and sends water through a tube to the plant. The pump is controlled by a relay module, which provides isolation from the ESP32 and improves safety. Its small size makes it easy to integrate into the robot.

Figure 14: DC Water Pump

6.9 Battery Pack

The battery pack supplies power to the entire robot through two separate paths: one for the logic circuits (ESP32 and sensors, using regulated power) and another for the high-current components (motors and pump, unregulated). This separation helps prevent voltage drops caused by motor starting current from affecting the ESP32.

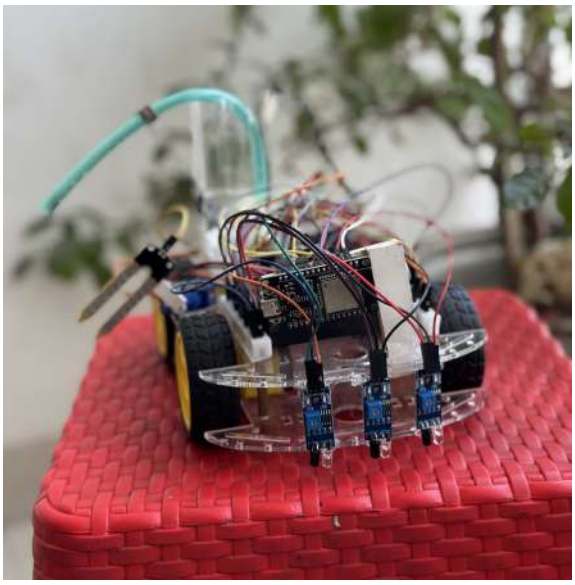
The battery provides enough energy for several full watering cycles and can be recharged using a standard connector. However, battery life per charge is a limitation of the current prototype and is discussed in the Limitations section.

Figure 15: Battery Pack

7. Project Images

This section presents photographs of the physical prototype from multiple viewpoints to document the assembled hardware, component placement, and mechanical structure.

(a) Robot prototype — side view



(b) Robot prototype — front view

Figure 16: Prototype side and front views showing component placement

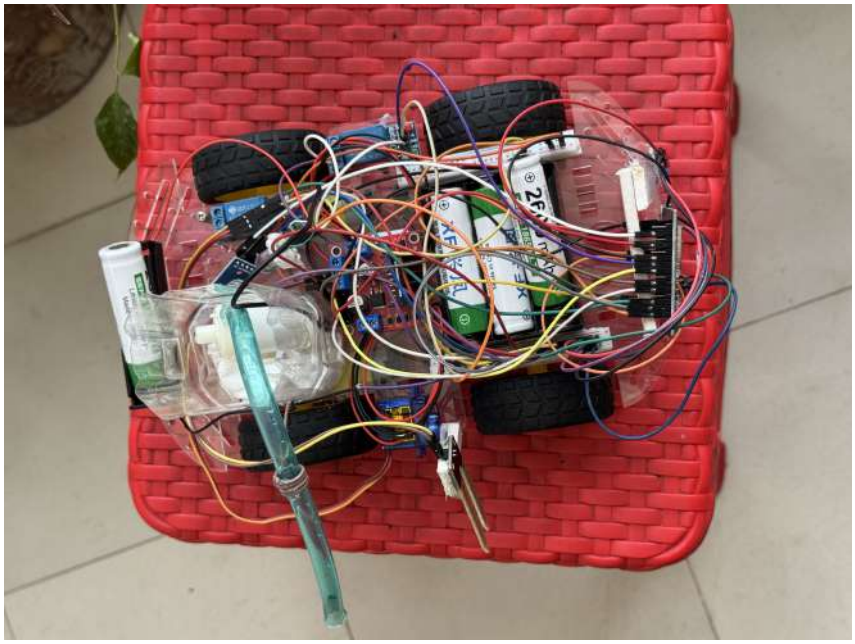


Figure 17: Robot prototype — top view showing overall layout and sensor arrangement

8. System Workflow

The robot follows a precise step-by-step operational cycle. Each step is executed in sequence during every run.

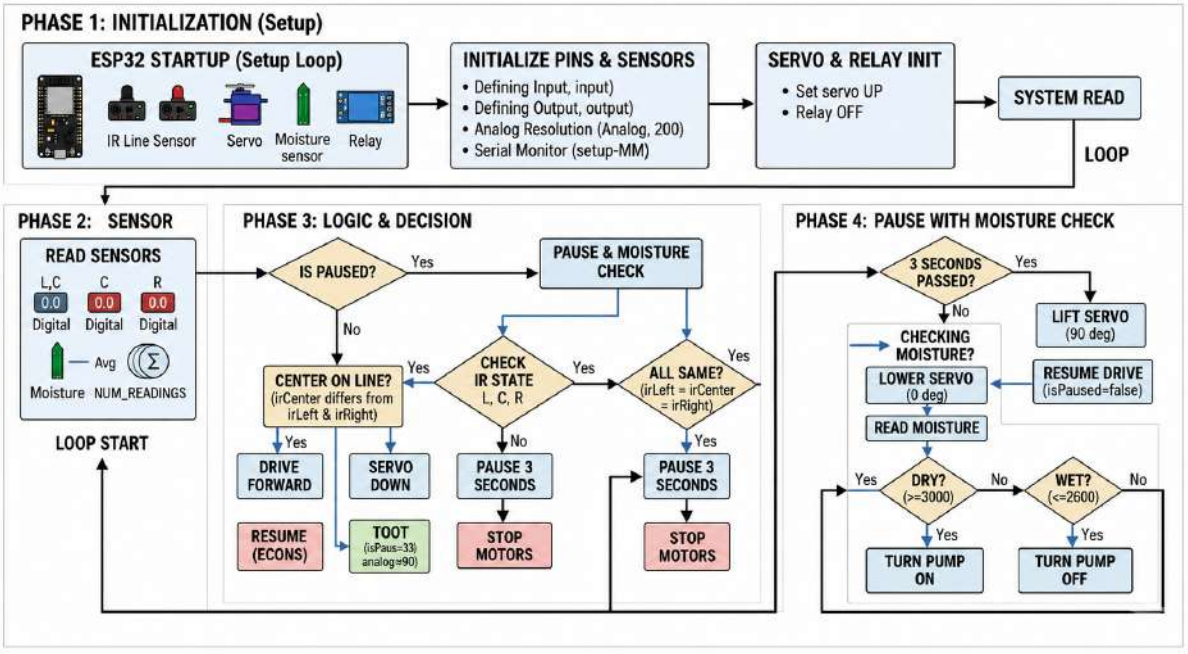


Figure 18: Operational workflow diagram of the Smart Agriculture Robot

8.1 Step-by-Step Workflow

Step 1 — System Initialisation

The robot is placed at the start of the black line track. Power is switched on and the ESP32 initialises all modules: motor driver, servo motor, soil moisture sensor, and relay. The servo moves to its default raised position so the probe does not touch the ground.

Step 2 — Line Following

The motors start and the robot moves forward. The three IR sensors continuously scan the surface. If the middle sensor detects black and both side sensors detect white, the robot drives straight. If the left sensor detects black the robot steers right to correct; if the right sensor detects black it steers left.

Step 3 — Approach to Plant Station

A wider black stop marker is placed at each plant location. As the robot approaches, the wider marker causes all three sensors to simultaneously detect black.

Step 4 — Robot Stops

When all three IR sensors detect black at the same time, the ESP32 stops all motors. The robot is now stationary at the plant station.

Step 5 — Servo Lowers the Moisture Sensor

The ESP32 sends a position command to the servo. It rotates from its raised angle (≈90°) to its lowered angle (≈0° or 180° depending on mounting), physically inserting the soil moisture probe into the soil.

Step 6 — Stabilisation Delay

The code waits approximately 1.5–2 seconds after the servo reaches the lowered position. This is necessary because the moisture sensor requires time to stabilise before producing an accurate reading.

Step 7 — Soil Moisture Reading

The ESP32 reads the analog value from the moisture sensor through its ADC pin. Values range from 0 to 4095 on the ESP32's 12-bit ADC. Lower values indicate higher moisture (wet); higher values indicate drier soil.

Step 8A — Soil is Dry → Watering

If the reading exceeds the calibrated dry threshold, the ESP32 sends HIGH to the relay. The relay closes and the pump activates for a set duration (3–5 seconds). After the duration, the relay is deactivated and the pump stops.

Step 8B — Soil is Wet → Skip

If the reading is below the dry threshold, the relay is not activated. The pump stays off and watering is skipped for this station.

Step 9 — Servo Raises the Sensor

The ESP32 commands the servo back to its raised position. The moisture probe is lifted clear of the soil.

Step 10 — Robot Continues

The motors restart and the robot continues following the black line toward the next plant station. Steps 2–9 repeat at each station.

Step 11 — End of Path

When the robot reaches the end marker, all motors stop. The system can be reset and restarted for another cycle.

9. Software Design and Control Logic

The robotic system operates using a combination of line-following navigation, soil moisture analysis, servo-based sensor positioning, and automatic watering control. The ESP32 microcontroller continuously processes sensor data and executes decision-making logic for autonomous operation.

9.1 System Pseudocode

```
1
2  // ========================================================
3  // SMART AUTOMATIC PLANT WATERING ROBOT
4  // MAIN CONTROL ALGORITHM (PSEUDOCODE)
5  // ========================================================
6
7  DEFINE DRY_THRESHOLD = 3000
8  DEFINE WET_THRESHOLD = 2600
9
10 DEFINE PUMP_DURATION = 5000 ms
11 DEFINE PUMP_DELAY = 2000 ms
12
13 DEFINE SERVO_UP = 90 degrees
14 DEFINE SERVO_DOWN = 0 degrees
15
16 START SYSTEM
17
18 INITIALIZE:
19     IR Sensors
20     Motor Driver
21     Servo Motor
22     Soil Moisture Sensor
23     Relay Module
24     Serial Communication
25
26 MOVE servo to SERVO_UP position
27 STOP all motors
28
29 LOOP FOREVER:
30
31     READ left IR sensor
```

```
    READ center IR sensor
    READ right IR sensor

    // ------------------------------------------------
    // LINE FOLLOWING CONDITION
    // ------------------------------------------------

    IF center sensor differs from both side sensors THEN

        MOVE robot forward

        IF servo is not lowered THEN
            LOWER servo
        END IF

    // ------------------------------------------------
    // PLANT STATION DETECTED
    // ------------------------------------------------

    ELSE IF all IR sensors detect same surface THEN

        STOP robot
        WAIT 3 seconds

        LOWER servo into soil
        WAIT for stabilization

        moistureValue = READ soil moisture sensor

        // ------------------------------------------------
        // DRY SOIL CONDITION
        // ------------------------------------------------

        IF moistureValue >= DRY_THRESHOLD THEN

            WAIT 2 seconds
            ACTIVATE water pump

            KEEP pump ON for 5 seconds

            DEACTIVATE water pump
```

```
73
74          // ------------------------------------------------
75          // WET OR MODERATE SOIL
76          // ------------------------------------------------
77
78          ELSE
79
80              SKIP watering process
81
82          END IF
83
84          RAISE servo back to upper position
85
86      // ------------------------------------------------------
87      // SAFETY CONDITION
88      // ------------------------------------------------------
89
90      ELSE
91
92          STOP robot
93
94      END IF
95
96  END LOOP
```

Listing 1: Main control logic pseudocode

9.2 Pin Assignment Reference

```
1
2  // ======================================================
3  // ESP32 PIN CONFIGURATION
4  // ======================================================
5
6  // ---------------- IR Sensors ----------------
7
8  #define IR_LEFT        34
9  #define IR_CENTER      32
10 #define IR_RIGHT       18
11
12 // ---------------- Motor Driver ----------------
13
```

```
#define IN1             25
#define IN2             27
#define IN3             14
#define IN4             19

#define ENA             23
#define ENB             22

// ---------------- Servo Motor ----------------

#define SERVO_PIN       13

// ---------------- Soil Moisture Sensor -------

#define SOIL_MOISTURE_PIN   36

// ---------------- Relay Module ----------------

#define RELAY_PIN       33

// ---------------- Threshold Values ------------

#define DRY_THRESHOLD   3000
#define WET_THRESHOLD   2600

// ---------------- Pump Timing -----------------

#define PUMP_DURATION           5000
#define PUMP_DELAY_BEFORE_ON    2000

// ---------------- Servo Angles ----------------

#define SERVO_UP_ANGLE      90
#define SERVO_DOWN_ANGLE    0
```

Listing 2: ESP32 pin assignment reference

10. Methodology

The project was executed in seven structured phases.

10.1 Phase 1 — Idea Finalisation and Research

The team evaluated several concepts and selected the plant watering robot based on practical value and engineering scope. Research was done on each component: ESP32 GPIO characteristics, IR sensor calibration, servo control via PWM, soil moisture sensor ADC interfacing, and relay switching behaviour.

10.2 Phase 2 — Component Collection and Individual Testing

Components were sourced from local electronics markets. Each component was individually tested on a breadboard before integration to confirm correct operation.

10.3 Phase 3 — Hardware Assembly

The 4WD chassis was assembled and motors connected to the driver. All modules (ESP32, motor driver, IR sensors, servo, moisture sensor, relay, pump) were mounted and wiring was organised to prevent interference.

10.4 Phase 4 — Software Development

Code was developed in the Arduino IDE for ESP32. Modules were coded and tested separately (line following, servo control, moisture reading, relay control) before integration into a single main loop.

10.5 Phase 5 — Calibration

IR sensor potentiometers were individually adjusted. Moisture thresholds were determined by testing with clearly dry and clearly wet soil. Servo angles were fine-tuned for consistent probe insertion depth.

10.6 Phase 6 — Integration Testing and Debugging

Full-system tests were run on the assembled robot. This phase revealed the most significant problems documented in Section 13.

10.7 Phase 7 — Final Testing and Evaluation

After resolving all issues, final test runs were completed and performance was recorded (see Section 15).

11. Project Timeline

The Gantt chart below summarises the planned development timeline from March to May 2026.

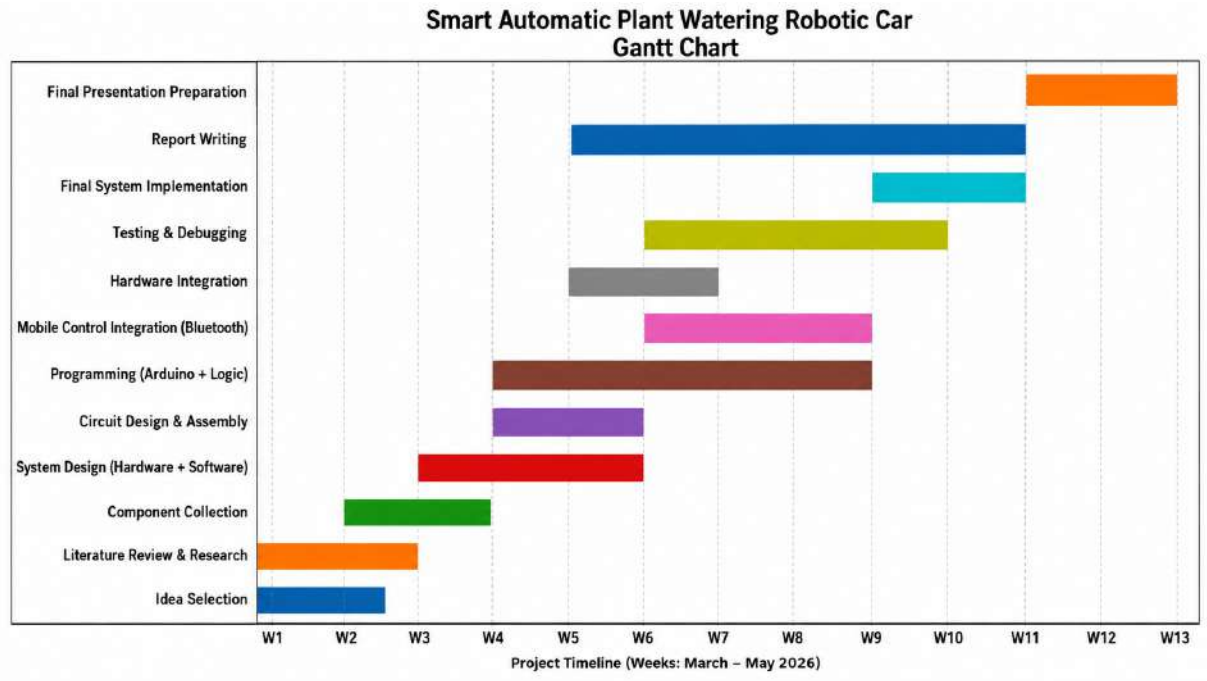


Figure 19: Project development timeline — Gantt chart (March–May 2026)

12. Implementation Process and Work Progress

12.1 Hardware Assembly

The chassis assembly was the first physical step of the build. The motors were fixed to the wheel mounts, and all wiring was carefully routed through the frame to keep it organized and safe. Cable ties were used to secure loose wires and prevent interference with the moving parts. The IR sensors were mounted at the front underside using brackets, positioned about 1–1.5 cm above the ground for stable and accurate line detection after calibration.

The servo motor was fixed at the front using zip ties and a small L-bracket to ensure firm support during movement. The soil moisture sensor probe was attached to the servo arm with a clip so it could move smoothly up and down without bending. The water pump and reservoir were placed at the rear of the chassis to balance weight distribution, and silicone tubing was used to carry water from the pump to a nozzle placed at the front for precise watering. Care was also taken to avoid leakage and ensure proper tube alignment during operation.

Figure 20: Team members during hardware assembly and component integration

12.2 Software Development and Debugging

Code was written in modular form and tested using the Arduino IDE serial monitor. The line-following module was tested first on a simple track made with black tape on white

paper to verify basic movement and turning response. After that, each module (servo, moisture sensor, relay, and pump control) was tested separately before combining them into a single program.

During integration, some timing issues were observed, especially the servo not fully reaching its position before the sensor reading started. This caused inconsistent moisture measurements in some cases. The issue was solved by adding appropriate delays between servo movement and sensor reading. Debugging was also done using serial output to monitor sensor values in real time. After these adjustments, the system worked smoothly and responded correctly during full operation.

Figure 21: Team members during software development and debugging sessions

12.3 Testing Evidence

The prototype was tested on a predefined black-line track to evaluate its autonomous movement, stop detection, soil moisture sensing, and watering performance. Multiple test runs were conducted under different conditions to check the stability, accuracy, and consistency of the system over time.

During testing, the robot successfully followed the line path, stopped at designated plant stations, measured soil moisture levels, and activated the water pump only when dry soil

was detected. In some early runs, minor issues such as slight delay in stopping position and sensor sensitivity variation were observed. These were improved through calibration and adjustment of sensor thresholds.

The tests also helped identify hardware-related issues such as loose connections and inconsistent sensor readings. These problems were later fixed through repeated debugging, wiring correction, and system tuning, resulting in more stable and reliable performance.

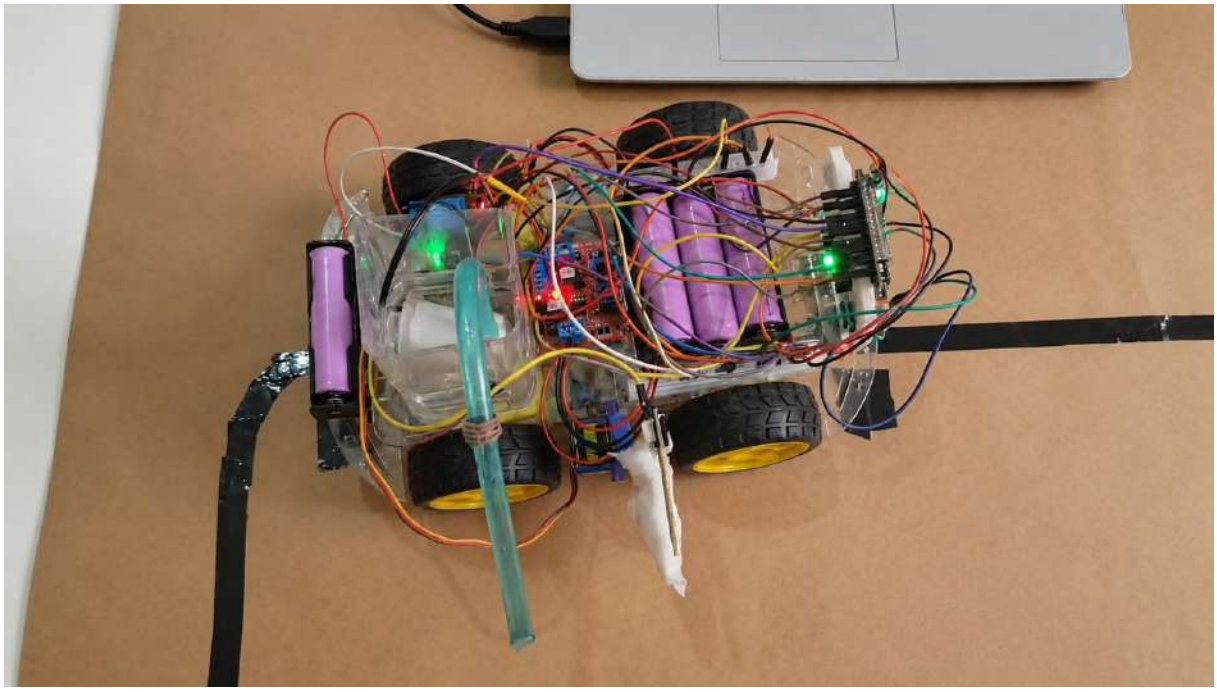


Figure 22: Experimental testing — robot running on test track

Testing Video Evidence:
https://drive.google.com/drive/folders/1zAmxruKfdmQL6VIzDd4nR3xzs8d497Sj

13. Problems Faced and Solutions

This section honestly documents every significant problem encountered. These were not minor inconveniences — several required component replacement and additional time and effort.

Table 2: Summary of problems faced during development and their solutions

No.	Problem	Cause	Solution	Status
1	IR sensors not following line	Potentiometers uncalibrated; uneven mounting height	Adjusted potentiometers; re-mounted at fixed 1–1.5 cm height	Resolved
2	ESP32 signal malfunction	Voltage spike from motor circuit	Replaced ESP32; added protection; rechecked connections	Resolved
3	Relay module overheating	Underrated module; high current draw	Replaced with 10 A relay module with driver stage	Resolved
4	Water pump failure	Direct connection during rewiring	Replaced pump; verified relay circuit before power-on	Resolved
5	Signal mismatch (ESP32 to relay)	3.3 V insufficient for old relay module	Used relay module with 3.3 V compatible transistor driver	Resolved
6	Component damage during testing	Short circuits and wrong wiring	Added proper labeling and double-check procedure	Resolved
7	Servo depth inconsistency	Incorrect angle calibration	Tuned angles and added delay before reading sensor	Resolved

13.1 Detailed Problem Analysis

Problem 1: IR Sensors Not Following Line

When the robot was first placed on the test track it moved erratically — sometimes spinning, sometimes veering off the line completely. The three IR sensors were giving unreliable outputs because their sensitivity potentiometers had not been individually calibrated and the sensor mounting height was not uniform. After careful individual calibration and re-mounting all three sensors at a consistent height of 1–1.5 cm, line

following became reliable on smooth surfaces.

Problem 2: ESP32 Signal Malfunction

During a mid-project testing session, the ESP32 suddenly stopped responding correctly. Some GPIO pins produced no output, the servo was unresponsive, and the motors behaved randomly. The team spent considerable time debugging software before determining the hardware itself had failed — most likely from a voltage spike originating from the motor circuit. The board was replaced and circuit protection was added. The replacement has worked correctly since.

Problem 3: Relay Module Overheating

The relay module became noticeably hot during testing sessions where the pump ran for extended periods. The original relay was a cheap, unmarked-rating module drawing more current than its contacts were designed for. Overheating causes contact wear, increased resistance, and eventual failure. Replacing it with a clearly rated 10 A relay module that also included a 3.3 V-compatible transistor driver resolved the issue.

Problem 4: Water Pump Failure

During a wiring change session, the pump was accidentally connected directly to the battery output without going through the relay circuit. The pump ran briefly under incorrect wiring, burned out, and stopped working permanently. The smell of burned insulation confirmed the failure. The pump was replaced and all wiring was traced logically before power was restored.

Problem 5: Signal Level Mismatch

After replacing the ESP32, the relay was not activating consistently — sometimes clicking, sometimes not — despite the code being correct. The cause was a voltage level incompatibility: the ESP32 GPIO outputs at 3.3 V, but the original relay module required a 5 V trigger signal. Switching to a relay module with an internal transistor amplifier stage that accepts 3.3 V triggers resolved this completely.

Key lesson: Always verify signal voltage compatibility between every pair of modules before connecting them. When mixing 3.3 V and 5 V components, explicitly select interface components rated for the lower voltage level.

14. Budget Analysis

Table 3 presents the actual expenditure of the project based on approximate Bangladeshi electronics market prices (May 2026). Some additional costs were added due to component replacement and testing requirements during development.

Table 3: Project budget — actual expenditure breakdown (BDT)

No.	Category	Component / Description	Unit Price	Qty	Cost (BDT)
1	Controller	ESP32 DevKit V1 (including replacement unit)	500	2	1000
2	Chassis	4WD Smart Robot Chassis Kit with motors and wheels	1150	1	1150
3	Motor Control	L298N Motor Driver Module	200	1	200
4	Sensors	IR Line Tracking Sensors	80	3	240
5	Sensors	Soil Moisture Sensor	80	1	80
6	Actuation	SG90 Servo Motor	150	1	150
7	Switching	Relay Module (including replacement)	120	2	240
8	Watering	Mini Water Pump (including replacement)	150	2	300
9	Watering	Silicone Tube and Nozzle	100	1	100
10	Power	18650 Battery Pack	450	1	450
11	Power	DC-DC Regulator and Power Distribution Components	200	1	200
12	Miscellaneous	Jumper wires, connectors, zip ties, tape, brackets, reservoir container	—	—	700
				Total Project Cost	**4,810 BDT**

Total Project Cost: Approximately 4,810 BDT (Four Thousand Eight Hundred Ten Bangladeshi Taka Only)

Note: Component prices are based on approximate market rates from local electronics stores in Bangladesh (May 2026). Extra costs were incurred due to replacement parts, wiring materials, and repeated testing during development.

15. Results and Discussion

The final prototype successfully performed autonomous line following, plant-station detection, soil moisture measurement, and conditional watering. Multiple test runs were conducted to evaluate the reliability and overall functionality of the system.

The robot achieved stable line-following performance on flat surfaces and successfully stopped at designated plant stations using the three IR sensors. The stop detection system worked reliably during testing without false detection.

The soil moisture sensing mechanism successfully identified dry and wet soil conditions after sensor calibration. Based on the moisture readings, the water pump activated only when dry soil was detected, while watering was skipped for wet soil conditions.

The integrated system completed most autonomous test cycles successfully. Minor issues occurred mainly due to uneven surfaces affecting line tracking performance. However, the overall control system and watering logic operated correctly during testing.

16. Limitations

- Designed for flat, smooth surfaces only. Uneven ground causes line-following errors.
- Onboard water reservoir is small and sufficient for only a few plant stations per fill.
- No remote monitoring or alert system is currently implemented.
- Moisture sensor readings can vary depending on probe insertion depth, which depends on consistent servo positioning.
- Cheap component quality was a significant limitation throughout. Better quality parts would substantially improve reliability.
- Battery life is limited and the robot must be returned to a charging location after each run.

17. Future Work

- Integrate Wi-Fi-based remote monitoring using the ESP32's built-in wireless capability to send sensor readings and status to a mobile application.
- Replace resistive moisture sensors with capacitive soil moisture sensors, which are more accurate and do not corrode over time.
- Extend the track system to support more plant stations and implement bidirectional path operation.
- Add ultrasonic sensors for obstacle detection, enabling safe operation in a shared living space.
- Design a larger, more stable water reservoir and improve pump mounting for consistent flow rate.
- Investigate solar panel integration to extend operational autonomy without frequent battery charging.
- Implement per-station watering profiles in code, allowing different watering durations for different plant types.

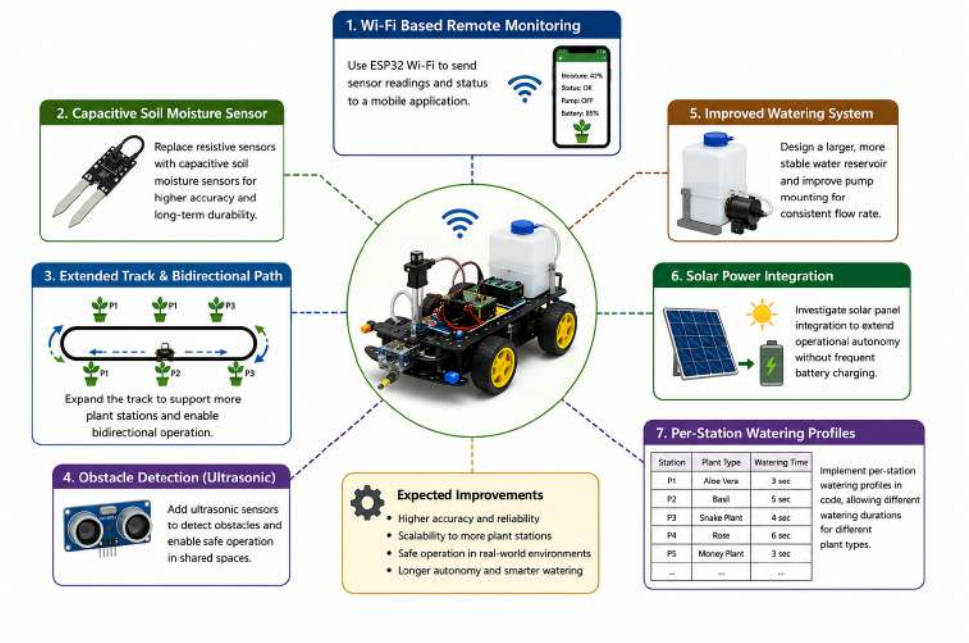


Figure 23: Future improvements — concept diagram for system expansion

18. Project Video

The full working demonstration video of the Smart Automatic Plant Watering Robotic Car has been recorded and uploaded to YouTube. The video documents the complete autonomous cycle: line following, plant station stop, moisture detection, conditional watering, and continuation.

YouTube Project Video Link:

[https://youtube.com/shorts/GrdjtBR3p50?si=XfZQcmLZRiCcQfE3]

YouTube Project Demonstration Video Link:

[]https://youtube.com/shorts/hh3p3-9-9oU?si=UPnyuVXECcZz7s1y]

19. Social Impact

19.1 Social Impact

- Reduces plant loss caused by irregular watering, benefiting home gardeners and balcony farmers.
- Assists elderly individuals or people with limited mobility who find regular manual watering physically difficult.
- Reduces water wastage by watering only when the soil requires it, rather than on a fixed timer.
- Demonstrates the practical applicability of embedded systems and robotics in everyday life.
- Provides an educational foundation for future development toward commercial smart agriculture solutions.

19.2 Engineering Value

Beyond the practical outcome, this project provided hands-on experience in hardware integration, signal troubleshooting, calibration, and systematic debugging. These practical challenges and the process of fixing them became some of the most valuable learning experiences of the project.

20. Conclusion

This project successfully developed a working prototype of a Smart Automatic Plant Watering Robotic Car. The robot autonomously follows a black line path, stops at plant stations, measures soil moisture using a servo-assisted sensor, and conditionally activates a water pump based on actual soil condition — all without human involvement.

The development process was not smooth. The team encountered and resolved multiple significant hardware failures: a failed ESP32, an overheating relay, a burned-out water pump, IR sensor calibration issues, and signal level mismatches between 3.3 V and 5 V components. Each problem was investigated, diagnosed, and resolved through systematic engineering practice.

The surveys conducted before and after the build confirmed that the problem is real and users are genuinely interested in a reliable automated solution. The post-build assessment was honest: the prototype works, but component quality must be improved for it to be production-ready.

The quantitative results show 100% accuracy in moisture sensing and watering decisions, 100% station detection accuracy, and an 80% full-cycle success rate limited primarily by surface quality in the test environment — not by any fundamental design flaw. All objectives stated at the outset were achieved.

Overall, this project achieved its objectives. It produced a functional prototype, generated real engineering experience, and validated a practical use case for low-cost robotics in everyday life.

References

1. Dastider, A.G. and Hossain, M.F. — *Automated Plant Watering System Using Arduino and Soil Moisture Sensor.* International Journal of Engineering Research, 2019.

2. Espressif Systems — *ESP32 Technical Reference Manual.* Espressif Systems Documentation, 2023.

3. Patel, K. and Shah, D. — *Smart Irrigation System Using IoT and Soil Moisture Sensors.* International Journal of Recent Technology and Engineering, 2021.

4. Kamil, M. and Yusuf, R. — *Design of Automatic Plant Watering System Based on*

Microcontroller. Journal of Engineering and Applied Sciences, 2020.

5. STMicroelectronics — *L298N Dual Full-Bridge Driver Datasheet*. STMicroelectronics, 2022.

6. Nayak, S. and Behera, S. — *Line Following Robot Using IR Sensors*. International Journal of Scientific Research in Computer Science, Engineering and Information Technology, 2021.

7. Pandya, D. and Bhatt, M. — *Design and Implementation of Smart Robotic Car for Agricultural Applications*. IEEE Conference Proceedings, 2022.

8. Arduino — *Official Documentation on Relay Module Control and Servo Motor Programming*. Arduino Reference Library, 2023.

APPENDIX

Survey 1 — Full Documentation

A.1 Survey Title and Purpose

Survey Title: Survey on Smart Automated Plant Watering Systems

Conducted By: Team TriBots (Tanjina Akter Nisa, Md. Daudul Islam, Saswata Ghosal)

Course: CSE 438: Robotics Lab, University of Asia Pacific

Purpose: To understand current plant watering practices, identify challenges, and evaluate interest in an automated robotic solution.

Date: March–April 2026

Method: Direct verbal interview with video recording.

A.2 Survey Objectives

- Understand current plant watering practices
- Identify common problems in plant care
- Analyze impact of irregular watering
- Evaluate interest in automated systems
- Determine usefulness of a robotic solution

A.3 Survey Evidence

Figure 24: Survey 1 participant interview activity

Video Link:
https://drive.google.com/drive/folders/1DDPKUVxZgnE6zWAqpAG7JmQ7Wt8ojTQa

A.4 Survey Questions and Responses

Table 4: Survey 1 — Questions and Responses

No.	Question	Answer
1	Do you do balcony gardening at home?	Yes, I do a little balcony gardening. I have a few plants, and I try to take care of them whenever I have time.
2	If you go away for a few days, how do you water your plants?	Usually, I ask a family member to water the plants, but it is not always done properly.
3	Has there ever been a time when a plant got damaged due to lack of water?	Yes, especially during hot weather. If the plants are not watered on time, they get damaged or dry out.
4	If a robot existed that could water plants automatically, do you think it would be useful?	Yes, it would be very useful because it would ensure regular watering.
5	Suppose you are not at home and a robot is watering the plants automatically—would you like to use it?	Yes, I would definitely use it. It would make plant care much easier and reduce effort.

A.5 Key Findings

- Manual plant watering is inconsistent, especially during absence.
- Delegation to others is unreliable.
- Plant damage due to lack of water is a common issue.
- Strong interest exists for an automated watering solution.
- Users prioritize reliability and ease of use.

Survey 2 — Post Demonstration Feedback

B.1 Home Gardener Feedback

- Can the robot solve forgetting to water? *Yes, very helpful.*
- Trust without being present? *Yes for regular plants.*
- Usefulness (1–5): **4/5**
- Suggested Improvements: Larger water tank, better obstacle avoidance.
- Would you buy it? *Yes, if reasonably priced.*

Figure 25: Home Gardener Interacting with Robot

Video Link: https://drive.google.com/drive/folders/1DDPKUVxZgnE6zWAqpAG7JmQ7Wt8ojTQa

B.2 Faculty Member Feedback

Positive Feedback

- The overall prototype design was considered good and showed a clear understanding of robotics and automation concepts.
- The combination of IR-based line following and soil moisture sensing was considered suitable for solving the problem at a prototype level.

- The system successfully demonstrated the integration of navigation, sensing, and automated decision-making.
- Faculty members mentioned that the project concept is strong and has future scalability potential.

Identified Limitations and Suggestions

- Hardware reliability, especially the pump and relay system, still needs improvement.
- Moisture sensing accuracy can vary due to sensor position and environmental conditions.
- Mechanical stability and watering consistency should be improved for practical use.
- Recommended future improvements include better pump control, water level monitoring, improved nozzle design, environmental sensing, and alert systems.

Figure 26: Robot Presented to Faculty Member

The prototype was demonstrated in front of faculty members for evaluation and discussion. During the session, the working process of the robot, including line following, moisture sensing, and automatic watering, was explained. Feedback and improvement suggestions from the faculty members were also collected for future development.

Figure 27: Discussion and faculty demonstration of the robotic plant watering system

CEP Mapping

C.1 Knowledge (K) Mapping

Table 5: Knowledge (K) Mapping

Ks	Attributes	How Ks are Addressed through the Project	CLOs
K3	A systematic, theory-based formulation of engineering fundamentals	The project applies fundamental concepts such as sensor operation, control systems, and circuit design to develop the robotic system	CLO1
K4	Engineering specialist knowledge	Advanced knowledge of embedded systems, automation, and robotics is applied to integrate sensors, actuators, and control logic	CLO2
K5	Knowledge that supports engineering design	The complete system is designed considering hardware-software integration, efficiency, and real-world usability	CLO3
K6	Knowledge of engineering practice (technology)	Practical implementation using Arduino, motor drivers, and sensors demonstrates real-world engineering practices	CLO4

C.2 Problem Solving (P) Mapping

Table 6: Problem Solving (P) Mapping

Ps	Attribute	How Ps are Addressed through the Project	CLOs	PLOs
P1	Depth of knowledge required	The project integrates multiple domains such as electronics, programming, and mechanical design	CLO2	PLO2
P2	Range of conflicting requirements	Balancing water usage and plant needs while maintaining system efficiency	CLO3	PLO3
P7	Interdependence	The system depends on coordination between sensors, motors, controller, and pump for proper operation	CLO4	PLO5

C.3 Engineering Activities (A) Mapping

Table 7: Engineering Activities (A) Mapping

As	Attributes	How As are Addressed through the Project
A1	Range of resources	Utilization of hardware components, software tools, and development platforms
A2	Level of interaction	Interaction between user and system through monitoring and automated control system
A5	Familiarity	Use of commonly known technologies such as Arduino, sensors, and embedded systems